

crewAIInc / crewAI

<> Code Issues 67 Pull requests 39 Discussions Actions Projects Security Insights

Framework for orchestrating role-playing, autonomous AI agents. By fostering collaborative intelligence, CrewAI empowers agents to work together seamlessly, tackling complex tasks.

[crewai.com](#)

MIT license

Security policy

30.9k stars 4.1k forks 292 watching Branches Activity Custom properties

Tags

Public repository

..

480 Branches

83 Tags

Go to file

Go to file

+

Add file

Code

vinibrsl

Identify parent_flow Of Crew and LiteAgent (#2723)

17474a3 · 2 days ago

.cache/plugin/social	adding manager_llm	last year
.github	pytest improvements to handle flaky test (...)	3 days ago
docs	fix: renaming TaskGuardrail to LLMGuardra...	4 days ago
src/crewai	Identify parent_flow Of Crew and LiteAgent...	2 days ago
tests	Identify parent_flow Of Crew and LiteAgent...	2 days ago
.editorconfig	adding editor config	last year
.gitignore	small comments	last month
.pre-commit-config.yaml	sort imports with isort rules by ruff linter (...)	5 months ago
.ruff.toml	sort imports with isort rules by ruff linter (...)	5 months ago
LICENSE	preparing new version	2 months ago
README.md	fix: add type hints and ignore type checks f...	3 weeks ago
crewAI.excalidraw	Changed "Inert" to "Innate" #470 (#490)	last year
mkdocs.yml	Add documentation for mlflow tracing inte...	3 months ago
pyproject.toml	pytest improvements to handle flaky test (...)	3 days ago
uv.lock	pytest improvements to handle flaky test (...)	3 days ago

https://github.com/crewAllnc/crewAI

1/16

Fast and Flexible Multi-Agent Automation Framework

CrewAI is a lean, lightning-fast Python framework built entirely from scratch—completely **independent of LangChain or other agent frameworks**. It empowers developers with both high-level simplicity and precise low-level control, ideal for creating autonomous AI agents tailored to any scenario.

- **CrewAI Crews:** Optimize for autonomy and collaborative intelligence.
- **CrewAI Flows:** Enable granular, event-driven control, single LLM calls for precise task orchestration and supports Crews natively

With over 100,000 developers certified through our community courses at learn.crewai.com, CrewAI is rapidly becoming the standard for enterprise-ready AI automation.

CrewAI Enterprise Suite

CrewAI Enterprise Suite is a comprehensive bundle tailored for organizations that require secure, scalable, and easy-to-manage agent-driven automation.

You can try one part of the suite the [Crew Control Plane for free](#)

Crew Control Plane Key Features:

- **Tracing & Observability:** Monitor and track your AI agents and workflows in real-time, including metrics, logs, and traces.
- **Unified Control Plane:** A centralized platform for managing, monitoring, and scaling your AI agents and workflows.
- **Seamless Integrations:** Easily connect with existing enterprise systems, data sources, and cloud infrastructure.
- **Advanced Security:** Built-in robust security and compliance measures ensuring safe deployment and management.
- **Actionable Insights:** Real-time analytics and reporting to optimize performance and decision-making.
- **24/7 Support:** Dedicated enterprise support to ensure uninterrupted operation and quick resolution of issues.
- **On-premise and Cloud Deployment Options:** Deploy CrewAI Enterprise on-premise or in the cloud, depending on your security and compliance requirements.

CrewAI Enterprise is designed for enterprises seeking a powerful, reliable solution to transform complex business processes into efficient, intelligent automations.

[Homepage](#) | [Documentation](#) | [Chat with Docs](#) | [Discourse](#)

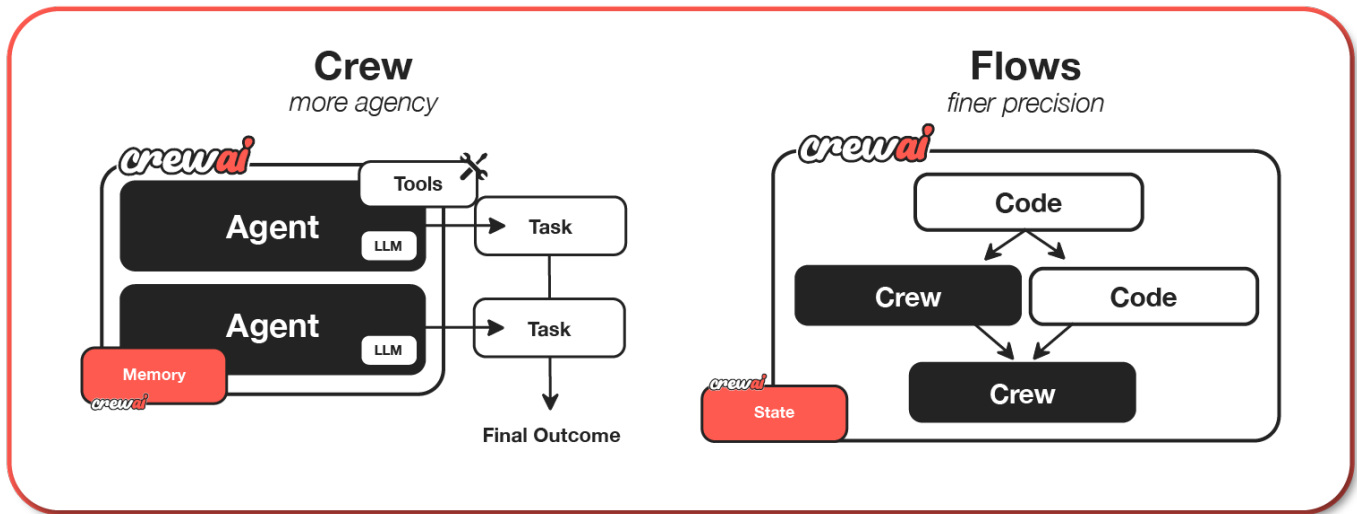


Table of contents

- [Why CrewAI?](#)
- [Getting Started](#)
- [Key Features](#)
- [Understanding Flows and Crews](#)
- [CrewAI vs LangGraph](#)
- [Examples](#)
 - [Quick Tutorial](#)
 - [Write Job Descriptions](#)
 - [Trip Planner](#)
 - [Stock Analysis](#)
 - [Using Crews and Flows Together](#)
- [Connecting Your Crew to a Model](#)
- [How CrewAI Compares](#)
- [Frequently Asked Questions \(FAQ\)](#)

- [Contribution](#)
- [Telemetry](#)
- [License](#)

Why CrewAI?



CrewAI unlocks the true potential of multi-agent automation, delivering the best-in-class combination of speed, flexibility, and control with either Crews of AI Agents or Flows of Events:

- **Standalone Framework:** Built from scratch, independent of LangChain or any other agent framework.
- **High Performance:** Optimized for speed and minimal resource usage, enabling faster execution.
- **Flexible Low Level Customization:** Complete freedom to customize at both high and low levels - from overall workflows and system architecture to granular agent behaviors, internal prompts, and execution logic.
- **Ideal for Every Use Case:** Proven effective for both simple tasks and highly complex, real-world, enterprise-grade scenarios.
- **Robust Community:** Backed by a rapidly growing community of over **100,000 certified** developers offering comprehensive support and resources.

CrewAI empowers developers and enterprises to confidently build intelligent automations, bridging the gap between simplicity, flexibility, and performance.

Getting Started

Learning Resources

Learn CrewAI through our comprehensive courses:

- [Multi AI Agent Systems with CrewAI](#) - Master the fundamentals of multi-agent systems
- [Practical Multi AI Agents and Advanced Use Cases](#) - Deep dive into advanced implementations

Understanding Flows and Crews

CrewAI offers two powerful, complementary approaches that work seamlessly together to build sophisticated AI applications:

1. **Crews:** Teams of AI agents with true autonomy and agency, working together to accomplish complex tasks through role-based collaboration. Crews enable:
 - Natural, autonomous decision-making between agents
 - Dynamic task delegation and collaboration
 - Specialized roles with defined goals and expertise
 - Flexible problem-solving approaches

2. **Flows:** Production-ready, event-driven workflows that deliver precise control over complex automations. Flows provide:

- Fine-grained control over execution paths for real-world scenarios
- Secure, consistent state management between tasks
- Clean integration of AI agents with production Python code
- Conditional branching for complex business logic

The true power of CrewAI emerges when combining Crews and Flows. This synergy allows you to:

- Build complex, production-grade applications
- Balance autonomy with precise control
- Handle sophisticated real-world scenarios
- Maintain clean, maintainable code structure

Getting Started with Installation

To get started with CrewAI, follow these simple steps:

1. Installation

Ensure you have Python ≥ 3.10 < 3.13 installed on your system. CrewAI uses [UV](#) for dependency management and package handling, offering a seamless setup and execution experience.

First, install CrewAI:

```
pip install crewai
```



If you want to install the 'crewai' package along with its optional features that include additional tools for agents, you can do so by using the following command:

```
pip install 'crewai[tools]'
```



The command above installs the basic package and also adds extra components which require more dependencies to function.

Troubleshooting Dependencies

If you encounter issues during installation or usage, here are some common solutions:

Common Issues

1. **ModuleNotFoundError: No module named 'tiktoken'**

- Install tiktoken explicitly: `pip install 'crewai[embeddings]'`
- If using embedchain or other tools: `pip install 'crewai[tools]'`

2. **Failed building wheel for tiktoken**

- Ensure Rust compiler is installed (see installation steps above)
- For Windows: Verify Visual C++ Build Tools are installed
- Try upgrading pip: `pip install --upgrade pip`
- If issues persist, use a pre-built wheel: `pip install tiktoken --prefer-binary`

2. Setting Up Your Crew with the YAML Configuration

To create a new CrewAI project, run the following CLI (Command Line Interface) command:

```
crewai create crew <project_name>
```



This command creates a new project folder with the following structure:

```
my_project/
├── .gitignore
├── pyproject.toml
├── README.md
├── .env
├── src/
│   └── my_project/
│       ├── __init__.py
│       ├── main.py
│       ├── crew.py
│       ├── tools/
│       │   ├── custom_tool.py
│       │   └── __init__.py
│       └── config/
│           ├── agents.yaml
│           └── tasks.yaml
```



You can now start developing your crew by editing the files in the `src/my_project` folder. The `main.py` file is the entry point of the project, the `crew.py` file is where you define your crew, the `agents.yaml` file is where you define your agents, and the `tasks.yaml` file is where you define your tasks.

To customize your project, you can:

- Modify `src/my_project/config/agents.yaml` to define your agents.
- Modify `src/my_project/config/tasks.yaml` to define your tasks.
- Modify `src/my_project/crew.py` to add your own logic, tools, and specific arguments.
- Modify `src/my_project/main.py` to add custom inputs for your agents and tasks.
- Add your environment variables into the `.env` file.

Example of a simple crew with a sequential process:

Instantiate your crew:

```
crewai create crew latest-ai-development
```



Modify the files as needed to fit your use case:

agents.yaml

```
# src/my_project/config/agents.yaml
researcher:
  role: >
    {topic} Senior Data Researcher
  goal: >
    Uncover cutting-edge developments in {topic}
  backstory: >
    You're a seasoned researcher with a knack for uncovering the latest
    developments in {topic}. Known for your ability to find the most relevant
    information and present it in a clear and concise manner.

reporting_analyst:
  role: >
    {topic} Reporting Analyst
  goal: >
    Create detailed reports based on {topic} data analysis and research findings
  backstory: >
    You're a meticulous analyst with a keen eye for detail. You're known for
    your ability to turn complex data into clear and concise reports, making
    it easy for others to understand and act on the information you provide.
```



tasks.yaml

```
# src/my_project/config/tasks.yaml
research_task:
  description: >
    Conduct a thorough research about {topic}
    Make sure you find any interesting and relevant information given
    the current year is 2025.
  expected_output: >
    A list with 10 bullet points of the most relevant information about {topic}
  agent: researcher

reporting_task:
  description: >
    Review the context you got and expand each topic into a full section for a report.
    Make sure the report is detailed and contains any and all relevant information.
  expected_output: >
    A fully fledged reports with the main topics, each with a full section of information.
    Formatted as markdown without '```'
  agent: reporting_analyst
  output_file: report.md
```

crew.py

```
# src/my_project/crew.py
from crewai import Agent, Crew, Process, Task
from crewai.project import CrewBase, agent, crew, task
from crewai_tools import SerperDevTool
from crewai.agents.agent_builder.base_agent import BaseAgent
from typing import List

@CrewBase
class LatestAiDevelopmentCrew():
    """LatestAiDevelopment crew"""
    agents: List[BaseAgent]
    tasks: List[Task]

    @agent
    def researcher(self) -> Agent:
        return Agent(
            config=self.agents_config['researcher'],
            verbose=True,
            tools=[SerperDevTool()]
        )

    @agent
    def reporting_analyst(self) -> Agent:
        return Agent(
            config=self.agents_config['reporting_analyst'],
            verbose=True
        )

    @task
    def research_task(self) -> Task:
        return Task(
            config=self.tasks_config['research_task'],
        )

    @task
    def reporting_task(self) -> Task:
        return Task(
            config=self.tasks_config['reporting_task'],
            output_file='report.md'
        )

    @crew
    def crew(self) -> Crew:
```

```

"""Creates the LatestAiDevelopment crew"""
return Crew(
    agents=self.agents, # Automatically created by the @agent decorator
    tasks=self.tasks, # Automatically created by the @task decorator
    process=Process.sequential,
    verbose=True,
)

```

main.py

```

#!/usr/bin/env python
# src/my_project/main.py
import sys
from latest_ai_development.crew import LatestAiDevelopmentCrew

def run():
    """
    Run the crew.
    """
    inputs = {
        'topic': 'AI Agents'
    }
    LatestAiDevelopmentCrew().crew().kickoff(inputs=inputs)

```



3. Running Your Crew

Before running your crew, make sure you have the following keys set as environment variables in your `.env` file:

- An [OpenAI API key](#) (or other LLM API key): `OPENAI_API_KEY=sk-...`
- A [Serper.dev](#) API key: `SERPER_API_KEY=YOUR_KEY_HERE`

Lock the dependencies and install them by using the CLI command but first, navigate to your project directory:

```

cd my_project
crewai install (Optional)

```



To run your crew, execute the following command in the root of your project:

```
crewai run
```



or

```
python src/my_project/main.py
```



If an error happens due to the usage of poetry, please run the following command to update your crewai package:

```
crewai update
```



You should see the output in the console and the `report.md` file should be created in the root of your project with the full final report.

In addition to the sequential process, you can use the hierarchical process, which automatically assigns a manager to the defined crew to properly coordinate the planning and execution of tasks through delegation and validation of results. [See more about the processes here.](#)

Key Features

CrewAI stands apart as a lean, standalone, high-performance framework delivering simplicity, flexibility, and precise control—free from the complexity and limitations found in other agent frameworks.

[README](#) [MIT license](#) [Security](#)

resource demands.

- **Flexible & Precise:** Easily orchestrate autonomous agents through intuitive [Crews](#) or precise [Flows](#), achieving perfect balance for your needs.
- **Seamless Integration:** Effortlessly combine Crews (autonomy) and Flows (precision) to create complex, real-world automations.
- **Deep Customization:** Tailor every aspect—from high-level workflows down to low-level internal prompts and agent behaviors.
- **Reliable Performance:** Consistent results across simple tasks and complex, enterprise-level automations.
- **Thriving Community:** Backed by robust documentation and over 100,000 certified developers, providing exceptional support and guidance.

Choose CrewAI to easily build powerful, adaptable, and production-ready AI automations.

Examples

You can test different real life examples of AI crews in the [CrewAI-examples repo](#):

- [Landing Page Generator](#)
- [Having Human input on the execution](#)
- [Trip Planner](#)
- [Stock Analysis](#)

Quick Tutorial



Write Job Descriptions

[Check out code for this example](#) or watch a video below:



Trip Planner

[Check out code for this example](#) or watch a video below:



Stock Analysis

[Check out code for this example](#) or watch a video below:



Using Crews and Flows Together

CrewAI's power truly shines when combining Crews with Flows to create sophisticated automation pipelines. CrewAI flows support logical operators like `or_` and `and_` to combine multiple conditions. This can be used with `@start`, `@listen`, or `@router` decorators to create complex triggering conditions.

- `or_` : Triggers when any of the specified conditions are met.
- `and_` Triggers when all of the specified conditions are met.

Here's how you can orchestrate multiple Crews within a Flow:

```
from crewai.flow.flow import Flow, listen, start, router, or_
from crewai import Crew, Agent, Task, Process
from pydantic import BaseModel

# Define structured state for precise control
class MarketState(BaseModel):
    sentiment: str = "neutral"
    confidence: float = 0.0
    recommendations: list = []

class AdvancedAnalysisFlow(Flow[MarketState]):
    @start()
    def fetch_market_data(self):
        # Demonstrate low-level control with structured state
        self.state.sentiment = "analyzing"
        return {"sector": "tech", "timeframe": "1W"} # These parameters match the task description template

    @listen(fetch_market_data)
    def analyze_with_crew(self, market_data):
        # Show crew agency through specialized roles
        analyst = Agent(
            role="Senior Market Analyst",
            goal="Conduct deep market analysis with expert insight",
            backstory="You're a veteran analyst known for identifying subtle market patterns"
        )
        researcher = Agent(
            role="Data Researcher",
```

```

goal="Gather and validate supporting market data",
backstory="You excel at finding and correlating multiple data sources"
)

analysis_task = Task(
    description="Analyze {sector} sector data for the past {timeframe}",
    expected_output="Detailed market analysis with confidence score",
    agent=analyst
)

research_task = Task(
    description="Find supporting data to validate the analysis",
    expected_output="Corroborating evidence and potential contradictions",
    agent=researcher
)

# Demonstrate crew autonomy
analysis_crew = Crew(
    agents=[analyst, researcher],
    tasks=[analysis_task, research_task],
    process=Process.sequential,
    verbose=True
)

return analysis_crew.kickoff(inputs=market_data) # Pass market_data as named inputs

@router(analyze_with_crew)
def determine_next_steps(self):
    # Show flow control with conditional routing
    if self.state.confidence > 0.8:
        return "high_confidence"
    elif self.state.confidence > 0.5:
        return "medium_confidence"
    return "low_confidence"

@listen("high_confidence")
def execute_strategy(self):
    # Demonstrate complex decision making
    strategy_crew = Crew(
        agents=[
            Agent(role="Strategy Expert",
                goal="Develop optimal market strategy")
        ],
        tasks=[
            Task(description="Create detailed strategy based on analysis",
                expected_output="Step-by-step action plan")
        ]
    )
    return strategy_crew.kickoff()

@listen(or_("medium_confidence", "low_confidence"))
def request_additional_analysis(self):
    self.state.recommendations.append("Gather more data")
    return "Additional analysis required"

```

This example demonstrates how to:

1. Use Python code for basic data operations
2. Create and execute Crews as steps in your workflow
3. Use Flow decorators to manage the sequence of operations
4. Implement conditional branching based on Crew results

Connecting Your Crew to a Model

CrewAI supports using various LLMs through a variety of connection options. By default your agents will use the OpenAI API when querying the model. However, there are several other ways to allow your agents to connect to models. For example, you can configure your agents to use a local model via the Ollama tool.

Please refer to the [Connect CrewAI to LLMs](#) page for details on configuring you agents' connections to models.

How CrewAI Compares

CrewAI's Advantage: CrewAI combines autonomous agent intelligence with precise workflow control through its unique Crews and Flows architecture. The framework excels at both high-level orchestration and low-level customization, enabling complex, production-grade systems with granular control.

- **LangGraph:** While LangGraph provides a foundation for building agent workflows, its approach requires significant boilerplate code and complex state management patterns. The framework's tight coupling with LangChain can limit flexibility when implementing custom agent behaviors or integrating with external systems.
P.S. CrewAI demonstrates significant performance advantages over LangGraph, executing 5.76x faster in certain cases like this QA task example ([see comparison](#)) while achieving higher evaluation scores with faster completion times in certain coding tasks, like in this example ([detailed analysis](#)).
- **Autogen:** While Autogen excels at creating conversational agents capable of working together, it lacks an inherent concept of process. In Autogen, orchestrating agents' interactions requires additional programming, which can become complex and cumbersome as the scale of tasks grows.
- **ChatDev:** ChatDev introduced the idea of processes into the realm of AI agents, but its implementation is quite rigid. Customizations in ChatDev are limited and not geared towards production environments, which can hinder scalability and flexibility in real-world applications.

Contribution

CrewAI is open-source and we welcome contributions. If you're looking to contribute, please:

- Fork the repository.
- Create a new branch for your feature.
- Add your feature or improvement.
- Send a pull request.
- We appreciate your input!

Installing Dependencies

```
uv lock
uv sync
```

Copy icon

Virtual Env

```
uv venv
```

Copy icon

Pre-commit hooks

```
pre-commit install
```

Copy icon

Running Tests

```
uv run pytest .
```

Copy icon

Running static type checks

```
uvx mypy src
```

Copy icon

Packaging

```
uv build
```



Installing Locally

```
pip install dist/*.tar.gz
```



Telemetry

CrewAI uses anonymous telemetry to collect usage data with the main purpose of helping us improve the library by focusing our efforts on the most used features, integrations and tools.

It's pivotal to understand that **NO data is collected** concerning prompts, task descriptions, agents' backstories or goals, usage of tools, API calls, responses, any data processed by the agents, or secrets and environment variables, with the exception of the conditions mentioned. When the `share_crew` feature is enabled, detailed data including task descriptions, agents' backstories or goals, and other specific attributes are collected to provide deeper insights while respecting user privacy. Users can disable telemetry by setting the environment variable `OTEL_SDK_DISABLED` to true.

Data collected includes:

- Version of CrewAI
 - So we can understand how many users are using the latest version
- Version of Python
 - So we can decide on what versions to better support
- General OS (e.g. number of CPUs, macOS/Windows/Linux)
 - So we know what OS we should focus on and if we could build specific OS related features
- Number of agents and tasks in a crew
 - So we make sure we are testing internally with similar use cases and educate people on the best practices
- Crew Process being used
 - Understand where we should focus our efforts
- If Agents are using memory or allowing delegation
 - Understand if we improved the features or maybe even drop them
- If Tasks are being executed in parallel or sequentially
 - Understand if we should focus more on parallel execution
- Language model being used
 - Improved support on most used languages
- Roles of agents in a crew
 - Understand high level use cases so we can build better tools, integrations and examples about it
- Tools names available
 - Understand out of the publicly available tools, which ones are being used the most so we can improve them

Users can opt-in to Further Telemetry, sharing the complete telemetry data by setting the `share_crew` attribute to `True` on their Crews. Enabling `share_crew` results in the collection of detailed crew and task execution data, including `goal`, `backstory`, `context`, and `output` of tasks. This enables a deeper insight into usage patterns while respecting the user's choice to share.

License

CrewAI is released under the [MIT License](https://github.com/crewAllInc/crewAI).

Frequently Asked Questions (FAQ)

General

- [What exactly is CrewAI?](#)
- [How do I install CrewAI?](#)
- [Does CrewAI depend on LangChain?](#)
- [Is CrewAI open-source?](#)
- [Does CrewAI collect data from users?](#)

Features and Capabilities

- [Can CrewAI handle complex use cases?](#)
- [Can I use CrewAI with local AI models?](#)
- [What makes Crews different from Flows?](#)
- [How is CrewAI better than LangChain?](#)
- [Does CrewAI support fine-tuning or training custom models?](#)

Resources and Community

- [Where can I find real-world CrewAI examples?](#)
- [How can I contribute to CrewAI?](#)

Enterprise Features

- [What additional features does CrewAI Enterprise offer?](#)
- [Is CrewAI Enterprise available for cloud and on-premise deployments?](#)
- [Can I try CrewAI Enterprise for free?](#)

Q: What exactly is CrewAI?

A: CrewAI is a standalone, lean, and fast Python framework built specifically for orchestrating autonomous AI agents. Unlike frameworks like LangChain, CrewAI does not rely on external dependencies, making it leaner, faster, and simpler.

Q: How do I install CrewAI?

A: Install CrewAI using pip:

```
pip install crewai
```



For additional tools, use:

```
pip install 'crewai[tools]'
```



Q: Does CrewAI depend on LangChain?

A: No. CrewAI is built entirely from the ground up, with no dependencies on LangChain or other agent frameworks. This ensures a lean, fast, and flexible experience.

Q: Can CrewAI handle complex use cases?

A: Yes. CrewAI excels at both simple and highly complex real-world scenarios, offering deep customization options at both high and low levels, from internal prompts to sophisticated workflow orchestration.

Q: Can I use CrewAI with local AI models?

A: Absolutely! CrewAI supports various language models, including local ones. Tools like Ollama and LM Studio allow seamless integration. Check the [LLM Connections documentation](#) for more details.

Q: What makes Crews different from Flows?

A: Crews provide autonomous agent collaboration, ideal for tasks requiring flexible decision-making and dynamic interaction. Flows offer precise, event-driven control, ideal for managing detailed execution paths and secure state management. You can seamlessly combine both for maximum effectiveness.

Q: How is CrewAI better than LangChain?

A: CrewAI provides simpler, more intuitive APIs, faster execution speeds, more reliable and consistent results, robust documentation, and an active community—addressing common criticisms and limitations associated with LangChain.

Q: Is CrewAI open-source?

A: Yes, CrewAI is open-source and actively encourages community contributions and collaboration.

Q: Does CrewAI collect data from users?

A: CrewAI collects anonymous telemetry data strictly for improvement purposes. Sensitive data such as prompts, tasks, or API responses are never collected unless explicitly enabled by the user.

Q: Where can I find real-world CrewAI examples?

A: Check out practical examples in the [CrewAI-examples repository](#), covering use cases like trip planners, stock analysis, and job postings.

Q: How can I contribute to CrewAI?

A: Contributions are warmly welcomed! Fork the repository, create your branch, implement your changes, and submit a pull request. See the Contribution section of the README for detailed guidelines.

Q: What additional features does CrewAI Enterprise offer?

A: CrewAI Enterprise provides advanced features such as a unified control plane, real-time observability, secure integrations, advanced security, actionable insights, and dedicated 24/7 enterprise support.

Q: Is CrewAI Enterprise available for cloud and on-premise deployments?

A: Yes, CrewAI Enterprise supports both cloud-based and on-premise deployment options, allowing enterprises to meet their specific security and compliance requirements.

Q: Can I try CrewAI Enterprise for free?

A: Yes, you can explore part of the CrewAI Enterprise Suite by accessing the [Crew Control Plane](#) for free.

Q: Does CrewAI support fine-tuning or training custom models?

A: Yes, CrewAI can integrate with custom-trained or fine-tuned models, allowing you to enhance your agents with domain-specific knowledge and accuracy.

Q: Can CrewAI agents interact with external tools and APIs?

A: Absolutely! CrewAI agents can easily integrate with external tools, APIs, and databases, empowering them to leverage real-world data and resources.

Q: Is CrewAI suitable for production environments?

A: Yes, CrewAI is explicitly designed with production-grade standards, ensuring reliability, stability, and scalability for enterprise deployments.

Q: How scalable is CrewAI?

A: CrewAI is highly scalable, supporting simple automations and large-scale enterprise workflows involving numerous agents and complex tasks simultaneously.

Q: Does CrewAI offer debugging and monitoring tools?

A: Yes, CrewAI Enterprise includes advanced debugging, tracing, and real-time observability features, simplifying the management and troubleshooting of your automations.

Q: What programming languages does CrewAI support?

A: CrewAI is primarily Python-based but easily integrates with services and APIs written in any programming language through

Releases 82

 **0.118.0** Latest
4 days ago

[+ 81 releases](#)

Packages

No packages published

Used by 13.8k



Contributors 242



[+ 228 contributors](#)

Deployments 453

 **github-pages** 4 days ago

[+ 452 deployments](#)

Languages

Python 99.8%

HTML 0.2%